

Næðingjökull

Coding 2 pdf 

Week 1:

→ stacks & queues are linear data structures

Stack

→ FIFO push() & pop()

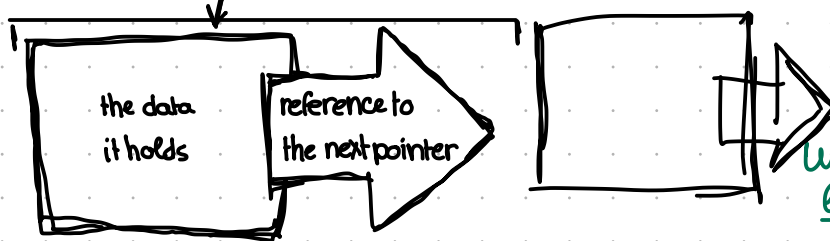
Queue

→ LIFO enqueue() & dequeue()

Week 2:

Linked lists → another linear data structure

→ consists of nodes:



Unlike Python's built-in lists, linked lists must be implemented manually using classes

Week 3 Linear data structure

Introduction to set theory & graph theory

Set theory → a group of unique items/values
NO 2 ITEMS CAN BE THE SAME

(re-visit Venn diagrams $n = \& \quad u = |$
with intersections (n) & unions (u))

graph theory → how nodes are connected by lines

example:

set theory → list of people

graph theory → map of connections between people

Week 4 Set theory & graph theory

Adjacency list $\rightarrow$ a data structure to represent a graph where each node in the graph stores a list of its neighbouring vertices

$\hookrightarrow$ another word for Node

Adjacency matrix $\rightarrow$ where each cell on index (i, j) stores an information about the edge from vertex i to vertex j

2D array $\hookleftarrow$

Week 5 - Graph Theory

The purpose of graphs is to encode relations on a finite set, specifically, Each vertex (NODE) on a graph is represented by a point - the graph is basically what connects them.

$\hookrightarrow$ I can basically use graphs to create a list of adjacencies $\rightarrow$ which NODE is connected to what, what there is in between each node, what connects them, etc

$\hookrightarrow$ a way to map data

Week 6 + 7 : Linear Algebra

Vectors $\rightarrow$ one dimensional array of lists
vector additions & multiplications & subtractions work

$$v_1 = [10, 1, 1, 12, 13]$$

$$v_2 = [1, 2, 3, 4]$$

$$v_3 = v_1 + v_2$$

$$\hookrightarrow = [11, 13, 15, 17]$$

Basic linear transformation $\rightarrow$ function that map a vector from one space to another

$$y = A \cdot x \rightarrow \text{an example of a basic linear transformation is scaling}$$

Week 8 - Numpy + intro to Task 2

dot product $\rightarrow$ mathematical operation that takes 2 vectors & combines them to produce a singular scalar value

Task 1 A Priority Queue for people's names

How (I understood the way) it works with the most important functions

```
def __init__(self, data):  
    initialising the queue
```

```
def peek(self):  
    returns highest  
    priority element
```

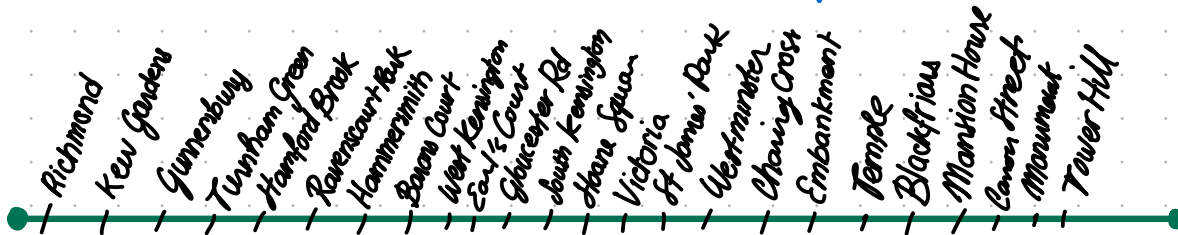
```
def add(self, data):  
    - adds element to the list  
    - sorts it by priority
```

How do I develop this concept as a task further?

The 1st task's concept is basically a self implemented sorting queue which rearranges the given data based on priority.

We were given another piece of code from the same tasks that uses this concept to sort TFL's train lines.

This is when I got the idea to create some sort of tube map route display



What will this code do?

You state from where on this line you are departing & where you want to go.

The code will show you the route of stations ahead of you.

I will try to use the function that organises by priority as a way to clarify the journey by distance

Through this I decided to also implement Node classes & apply graph theory

district line

Richmond ↔ Tower Hill

using Node classes,
linked lists,
stacks, queues, &
set theory

Task 1 → What would each function be used for?

STATIONS represented by nodes in a graph/ linked list

GRAPH adjacency with distances between consecutive stations

Priority-Queue adapted from the uploaded code to or
setting this up 1st distance of departure

SET THEORY visited set to avoid revisiting nodes

LINKED-LIST the district line is a linked list of nodes = stations

When we got to weeks 6 & 7 I was struggling to see how I can incorporate linear algebra into the already existing code